

Software Maturity Assessment — kpolimis.github.io

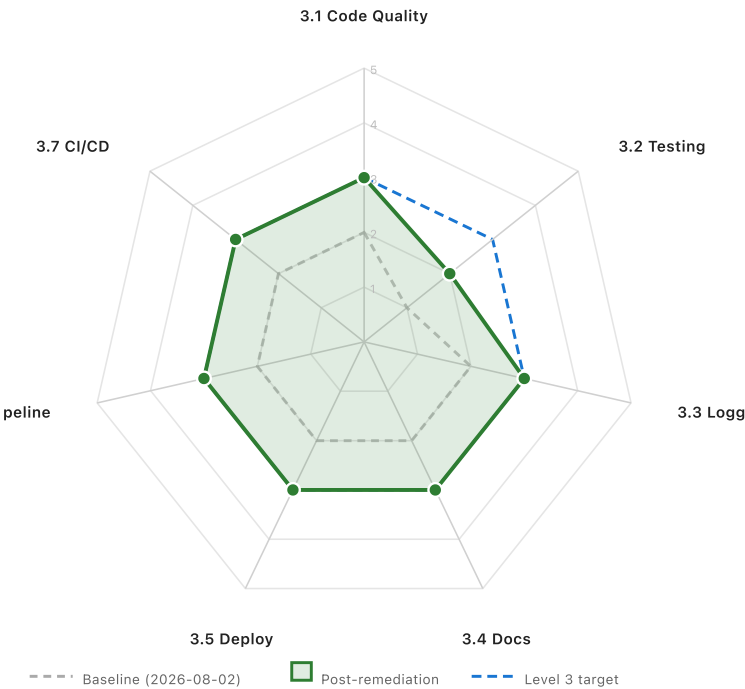
Date: 2026-08-02 (re-assessment) · **Baseline date:** 2026-08-02 · **Framework:** Atlas Analytics Internal Framework v1.0 · **Assessor:** Claude Sonnet 4.6 (automated, evidence-based) · **Tier:** Small / scripts-only · **Tier justification:** Quarto static blog with Python data-viz notebooks and one data-fetch script; Docker and full CI/CD are not required at this tier.

EXECUTIVE SUMMARY

Eight remediation items were completed in a single session immediately following the baseline assessment. Six of seven dimensions moved from Level 2 (Documented) to Level 3 (Automated): Ruff now enforces style on every push, pipe-delimited logging is consistent, docstrings cover all public functions, the bootstrap path is reproducible, the pipeline is orchestrated end-to-end via `make all`, and a lint+test CI gate blocks bad pushes. Testing moved from Level 1 to Level 2: 12 pytest tests exist and run in CI, but Pydantic/Pandera schema validation and coverage tracking are not yet in place. **Theme: successfully crossed the Documented → Automated threshold on six of seven dimensions in one sprint.**

Mean shown for internal trend orientation only; per methodology §2 it is not a primary score — read the radar. Baseline mean: 1.86 → Post-remediation mean: 2.86

SCORECARD & RADAR



Dimension	Previous	Now	Δ	Level
3.1 Code Quality & Style	2/5	3/5	+1	Automated
3.2 Testing & Validation	1/5	2/5	+1	Documented
3.3 Observability & Logging	2/5	3/5	+1	Automated
3.4 Documentation	2/5	3/5	+1	Automated
3.5 Deployment & Environment	2/5	3/5	+1	Automated
3.6 Data Pipeline / MLOps	2/5	3/5	+1	Automated
3.7 CI/CD & Automation	2/5	3/5	+1	Automated

Dashed grey polygon = baseline. Green polygon = post-remediation
Dashed blue ring = Level 3 target.

Level	Name	Meaning
1	Ad Hoc	Absent; individual habit only.
2	Documented	Written down, not enforced.
3	Automated	Enforced by tooling; violations caught.

4	Monitored	Automated + tracked over time.
5	Optimized	Automated, monitored, gates decisions.

DIMENSION FINDINGS

3.1 Code Quality & Style — 3/5 (Automated) was 2/5

Evidence: `pyproject.toml` added with Ruff config: E/F/I/W/C90/UP/B/D rules, Google docstring convention, max-complexity=10. `ruff check` passes with zero violations on `fetch_wc_data.py`. CI runs Ruff before every render (`publish.yml`). Type hints remain on all public signatures.

Gap to 4/5: Add `mypy` or `pyright` to CI; track lint-error count release over release.

3.2 Testing & Validation — 2/5 (Documented) was 1/5

Evidence: `tests/test_fetch.py` created with 12 tests across 3 classes (`TestStripRankPrefix` , `TestBuildLeagueStats` , `TestBuildEplClubStats`). All pass. Tests run in CI on every push to main. No coverage measurement; no Pydantic/Pandera at runtime boundaries (schema check is custom column-presence assertion, not a typed schema).

Gap to 3/5: Add `pytest-cov` to CI with a minimum coverage threshold; add Pandera schema validation at the three CSV load points (replacing the current custom check).

3.3 Observability & Logging — 3/5 (Automated) was 2/5

Evidence: `fetch_wc_data.py` uses `logging.getLogger(__name__)` throughout. Format is now pipe-delimited: `%(asctime)s | %(levelname)-8s | %(name)s | %(message)s` — matches SMALL_PROJECT_STANDARDS §7.3 exactly. Every filter/aggregation step logs a record count.

Gap to 4/5: Centralize log config in a `configure_logging()` function; ship logs to a store (even a rotating file); add a basic dashboard or log-grep workflow for error rate.

3.4 Documentation — 3/5 (Automated) was 2/5

Evidence: All six public functions in `fetch_wc_data.py` have Google-style docstrings with `Args:` / `Returns:` . All three `.qmd` helpers (`_colour` , `_logo` , `league_colour`) now have one-line docstrings. README expanded with first-time setup section, bootstrap instructions, and full Makefile target reference. Ruff D-rules enforce Google convention on every CI run.

Gap to 4/5: Add docstring coverage to CI reporting; version docs alongside code (a PR without updated docstrings should not pass review).

3.5 Deployment & Environment — 3/5 (Automated) was 2/5 (Small-tier; Docker N/A)

Evidence: `requirements.txt` complete with all pinned deps including `soccerdata==1.9.1` . `make bootstrap` target documents and executes the `footy` editable install. README first-time setup section gives a reproducible onboarding sequence. A fresh clone can now be set up with three commands: `pip install -r requirements.txt` , `make bootstrap` , `quarto preview` .

Gap to 4/5: Export the conda environment (`conda env export > environment.yml`) to pin the Python runtime alongside package versions; track image/env build times.

3.6 Data Pipeline / MLOps — 3/5 (Automated) was 2/5

Evidence: `make all` now wires the full pipeline: `fetch` → `render-wc-leagues` → `render-wc-epl`. Column-presence schema guards added in both `.qmd` setup cells — a missing or renamed column raises a `ValueError` immediately on load rather than propagating as silent `NaN` into charts. Custom check rather than Pandera, but the stage-boundary contract is now enforced.

Gap to 4/5: Replace custom checks with Pandera schemas; add null-rate and row-count logging between stages; version the CSV outputs.

3.7 CI/CD & Automation — 3/5 (Automated) was 2/5 (Small-tier)

Evidence: `publish.yml` now runs: (1) install ruff + pytest + pandas, (2) `ruff check` on `fetch_wc_data.py`, (3) `pytest tests/ -v`, then (4) Quarto render + gh-pages deploy. A lint or test failure blocks publish. Runs on every push to `main` (direct-to-main workflow; equivalent to PR-gating for a single-contributor project).

Gap to 4/5: Add `mypy` type-check step; track CI run time and flakiness; add branch protection requiring CI green before merge if collaborators are added.

PRIORITIZED ROADMAP — NEXT PHASE

P	Dimension	Action	Effort	Score Δ
P1	3.2 Testing (2→3)	Add <code>pytest-cov</code> with minimum threshold; replace custom schema check with Pandera in <code>.qmd</code> setup cells	2 hr	+1
P2	3.1 Quality (3→4)	Add <code>mypy --strict</code> to CI; track lint-error count per release	1 hr	+1
P3	3.5 Deploy (3→4)	Export <code>conda env export > environment.yml</code> ; document Python version pin	30 min	+1
P4	3.7 CI/CD (3→4)	Add <code>mypy</code> step to <code>publish.yml</code> ; track CI run-time trend	30 min	+1
P5	3.6 Pipeline (3→4)	Add Pandera schemas; log null rates + row counts at each stage boundary	2 hr	+1

APPENDIX — REMEDIATION EVIDENCE

Commit	Change	Dimension impact
922326b	Pipe-delimited log format in <code>fetch_wc_data.py</code>	3.3: 2→3
ccce9a7	Makefile: <code>all</code> , <code>bootstrap</code> , <code>lint</code> targets	3.5, 3.6: toward 3
e717429	Docstrings on <code>_colour</code> , <code>_logo</code> , <code>league_colour</code>	3.4: toward 3
58a5ad6	Pin <code>soccerdata==1.9.1</code> ; footy bootstrap in README	3.5: 2→3
285ef20	<code>pyproject.toml</code> with Ruff config; zero violations	3.1: 2→3
d19ea3b	Ruff lint gate in <code>publish.yml</code>	3.7: toward 3
71f6200	12 pytest tests; test gate in <code>publish.yml</code>	3.2: 1→2; 3.7: 2→3
0e1f3f7	Column-presence schema guards in both <code>.qmd</code> setup cells	3.6: 2→3